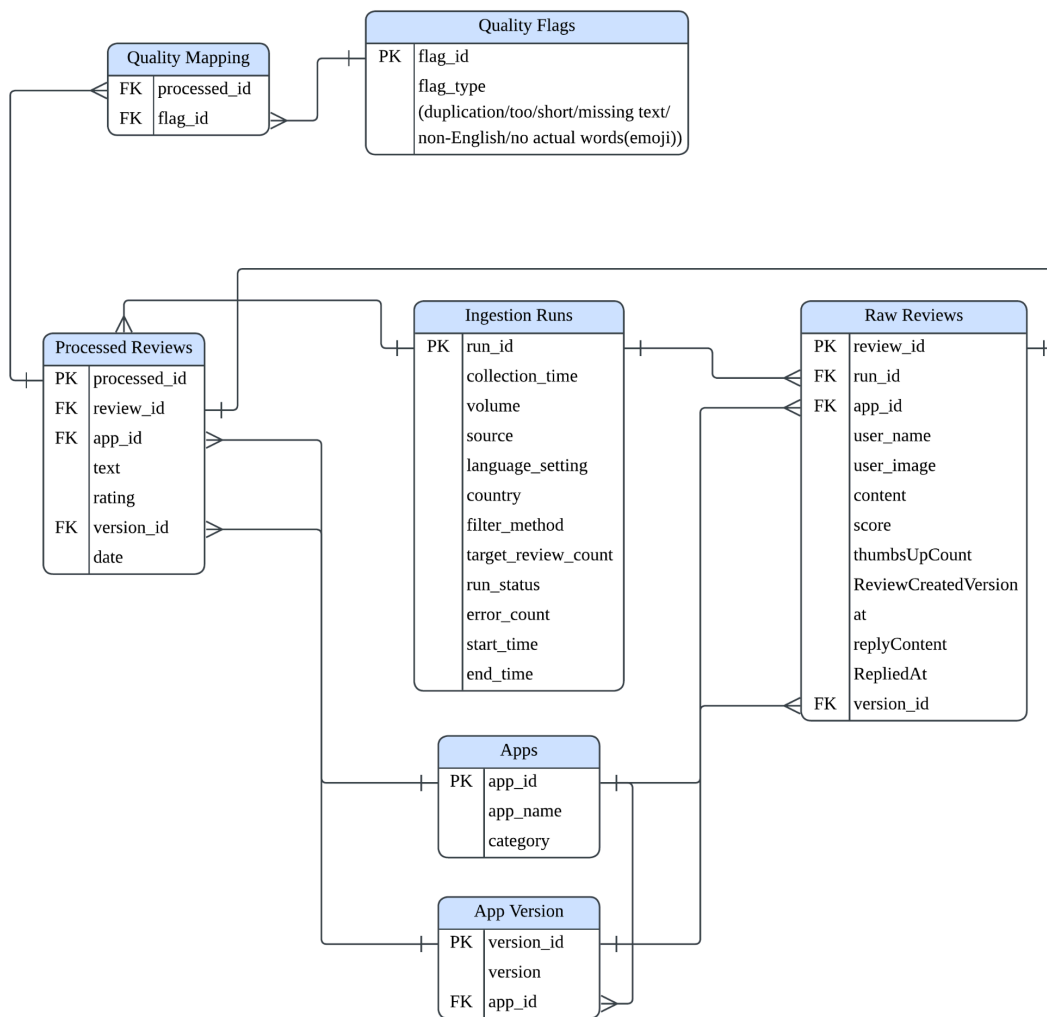




I got 7 tables shown above and have marked primary key and foreign key in the table.

Ingestion Runs: we only need run_id to uniquely identify which ingestion run the run belongs to, so I set it to primary key. Besides the basic collection time and volume, we want more operational text to clearly record details of each run, so I added source, language, country, filter_method, target_review_count, run_status, error_count, start_time and end_time to the table.

Apps: We only need app_id to uniquely identify each app, so app_id is the primary, and app_name and category gives more information about the specific app.

App Version: The primary key is version_id. In case of different app may share the same version, I added app_id as a foreign key to this table.

Raw Reviews: this table contains all information that can be scraped through the Google Play Store Scraper and also includes a primary key, review_id, to uniquely identify each review, while also including 3 foreign keys to connect the three tables above and one foreign key to the processed review table.

Processed Reviews: this table contains less information than the Raw Reviews table does because we only need useful information relevant to reviews, rather than user identifier. The primary key is processed_id, and I also include 3 foreign keys to connect the three basic tables, with one relationship connected to the raw reviews table, which is a one-to-one relationship.

Quality Flags: this table records all types of quality issues a review could potentially have and separating the quality flags into an independent table makes it more flexible for later expansion and clear analysis. The primary key is flag_id, and each id represents a type of quality issue.

Quality Mapping: since the relationship between Quality Flags table and Processed Reviews table is a many-to-many relationship. We need to add another table between them to make it a one-to-many relationship, so I added this table and included primary keys for two tables in it.

All relationships are one-to-many relationships, except the relationship between Processed Reviews and Raw Reviews because one raw review corresponds to only one processed review. For one-to-many relationship, we got the following logic:

- Every ingestion run can generate multiple raw reviews, so is the processed reviews table
- Every app can have multiple raw reviews, and then processed reviews
- Every app version can be related to many raw reviews, so is the processed reviews
- One app can have multiple versions

Deduplication Logic

1. Raw data deduplication key: Composite unique key (run_id, app_id, review_id) to eliminate duplicate scraped reviews from the same ingestion run and app.
2. Quality flags: Remove boolean columns (is_duplicated, low_signal, missing text) from Processed Reviews table. All quality issues are tracked through the Quality Flags table and QualityMapping one-to-many table for scalable tagging and detailed issue notes.

Tradeoffs

1. I separate the raw reviews and processed reviews into two tables so we can always access the raw dataset if we encounter any problems with the processed dataset. Even though it takes more space to store, I think reproducibility or traceability is more important than storage.
2. I include quality issues in a separate table to make it flexible for tracking and later expansion. Even though this new table adds more complexity for joining tables, flexibility is more important for traceability.
3. Even though it might increase query complexity to relate two reviews tables many times with the three tables in the middle, this provides a more clear and maintainable database structure (less repeated rows and keeps the schema lightweight and extensible with a great number of categories)